WattWise: A Regression-based Household Appliance Energy Consumption Prediction, with a test on Temporal Generalization

Swayam Shakya
Faculty of Science & Technology
Algoma University
Brampton, ON, Canada
swayamshakya000@gmail.com

***Abstract*—Household energy models require explicit prediction tasks and chronological evaluation to assess performance on later observations. This study develops WattWise, a reproducible Jupyter Notebook workflow that distinguishes estimation from recorded environmental conditions from forecasting with previously observed energy. Experiments use 19,735 ten-minute measurements from the Appliances Energy Prediction dataset. The first 80 percent of observations support fitting and temporal model selection; the remaining 3,947 observations form a later evaluation period. Environmental multiple linear regression with calendar features attains a root mean squared error of 85.69 watt-hours, and ridge regression attains 84.71 watt-hours. An expanded environmental search selects extra trees during development but performs worse on the final period, at 91.19 watt-hours. A separate history-based extra-trees forecast achieves 57.23 watt-hours, improving on persistence by 13.86 percent and on the reference lagged ridge by 6.17 percent. Three nested temporal validation folds also favor the forecasting selection procedure over persistence. The final period had already been inspected before the expanded study, so its later results are retrospective. A fresh audit reproduces 23 archived result artifacts within numerical tolerance, and all 49 automated tests pass. The evidence supports the usefulness of recent energy history while exposing weak environmental transfer, missed demand peaks, and the limits of inference from one dwelling.**

Keywords—Appliance energy, regression, generalization. time-series forecasting, temporal validation, reproducibility.

I. Introduction

Predicting household appliance energy is a practical regression problem in which the available information matters as much as the learning algorithm. Temperature, humidity, and weather measurements describe conditions associated with consumption, but they do not directly observe every appliance switch. Recent meter readings provide a different source of information: they describe the demand process immediately before the interval being predicted. Treating these two settings as interchangeable can overstate what a model has learned.

The Appliances Energy Prediction dataset provides an accessible, documented case study of this distinction [1]. Its originating research investigates several regression methods and combinations of household and weather measurements [2], with data and analysis scripts released by the authors [3]. WattWise revisits the problem using Python, explicit chronological evaluation, and an auditable link between fitted models, exported predictions, and reported results.

The study addresses three questions. First, does multiple linear regression (MLR) estimate later appliance energy more accurately than constant baselines under an environmental information constraint? Second, do calendar features, regularization, or a broader nonlinear search improve temporal transfer? Third, how much does previously observed energy help a separately defined ten-minute forecasting task? Root mean squared error (RMSE) is the selection criterion; mean absolute error (MAE) and the coefficient of determination provide complementary assessments.

The contribution is a reproducible benchmark and an integrated software artifact, rather than a new learning algorithm. It combines temporal validation, a declared follow-up search, residual and peak-demand diagnostics, dependence-aware uncertainty summaries, usable prediction interfaces, and executable verification. Both successful and unsuccessful changes are retained. The released evidence allows readers to inspect how the information available to a model and the evaluation protocol affect its measured performance.

II. Related Work and Study Scope

Candanedo, Feldheim, and Deramaix compare MLR, support vector regression, random forests, and gradient boosting for appliance energy using environmental measurements and lighting energy [2]. Their study establishes the dataset and motivation, but the present work uses a different predictor policy and temporal protocol. In particular, concurrent lighting energy is excluded here, and the main

evaluation follows the development observations in time. Published scores from the originating study are therefore contextual evidence, not a directly comparable leaderboard.

Ridge regression adds a squared-coefficient penalty to least squares and provides a controlled response to correlated predictors [4]. Extremely randomized trees introduce randomization in candidate features and split thresholds [5]. These established methods provide useful contrasts with MLR without requiring a novel estimator. Additive spline regression and histogram gradient boosting broaden the comparison while keeping the search small enough to reproduce on a conventional computer.

Model selection and performance estimation require separate treatment. Varma and Simon show that reporting the same cross-validation score used to optimize a model can bias the estimated error, and discuss nested validation as a remedy for that selection step [6]. WattWise uses expanding temporal folds at both levels. This evaluates a specified selection procedure within the development period; it does not establish unbiased performance across households, seasons, or researcher decisions made during the project.

The implementation uses scikit-learn pipelines and estimators [7]. The research emphasis is on transparent experimental conditions: every model must be interpreted together with its information set, training interval, selection rule, and forecast horizon. No claim of superiority over the originating paper or of a universally optimal model is made.

III. Data and Problem Formulation

A. Source Data and Provenance

The bundled source contains 19,735 rows and 29 columns, recorded from January 11, 2016 at 17:00 through May 27, 2016 at 18:00. Timestamps are unique, ordered, and separated by ten minutes. The target, Appliances, is energy in watt-hours (Wh) per observation interval, rather than instantaneous power. Multiplying interval energy by six would yield average watts over that interval; no such conversion is applied to the reported errors.

The source represents one Belgian dwelling over approximately four and a half months. It includes temperature and humidity sensors, weather measurements, lighting energy, and two supplied random variables. The dataset documentation identifies airport weather as hourly measurements interpolated to the observation cadence [1]. This matters when interpreting access to same-interval covariates. The raw CSV is redistributed under CC BY 4.0, separately from the project's MIT software license. Its SHA-256 is recorded in data/provenance.json and checked by an automated test.

The loader rejects absent required columns, missing or non-finite measurements, duplicate or unordered timestamps, cadence gaps, negative target values, and inputs with fewer than two observations. These checks confirm the supplied benchmark without silently repairing it. No target imputation, outlier removal, or synthetic training observations are used. The observed upper tail is part of the prediction problem and remains in all evaluations.

Fig. 1 summarizes only the development observations. The concentration at low energy and the extended upper tail motivate reporting absolute and squared-error metrics together. The hourly profile shows periodic structure but is descriptive: it does not identify an occupancy mechanism or a causal effect of time of day.

B. Two Distinct Prediction Tasks

The environmental task estimates energy for an interval using conditions recorded for that interval. Its 24 predictors are the nine temperature and humidity pairs T1–T9 and RH_1–RH_9, together with T_out, Press_mm_hg, RH_out, Windspeed, Visibility, and Tdewpoint. T6 and RH_6 describe the exterior north side of the dwelling; the remaining paired sensors describe indoor locations. The target, lights, rv1, rv2, and untransformed date are excluded. This is a retrospective estimation task, not an operational weather forecast.

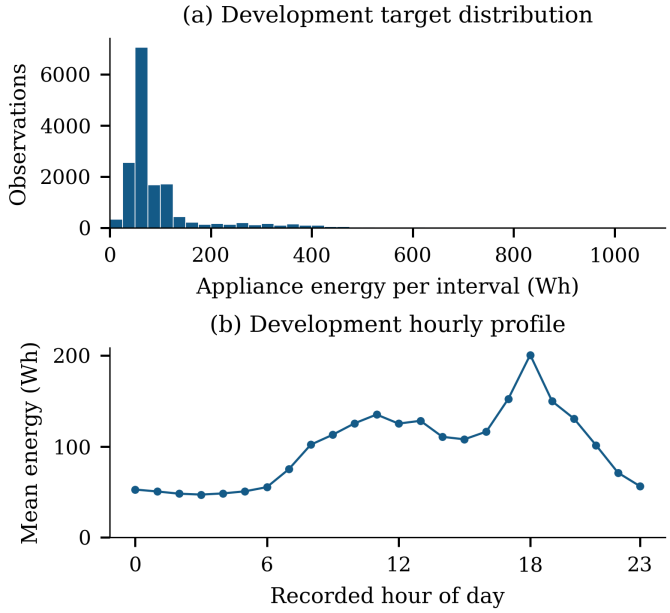


Fig. 1. Development data only: (a) distribution of all 15,788 target observations; (b) mean interval energy by recorded hour. High-use observations are retained. The hourly means are descriptive, without confidence intervals.

Calendar variants add sine and cosine encodings of hour-of-day and position within the week, plus a weekend indicator. Fractional hour uses minutes, and fractional week uses weekday plus fractional day. The environmental calendar design therefore contains 29 predictors. A monotonic date trend and month index are not added. The recorded timestamps are used as supplied; no unverified timezone or daylight-saving correction is introduced.

The forecast task predicts the next interval from observations available through the preceding interval and known calendar values. It excludes contemporaneous environmental measurements. The reference forecast uses target lags of 1, 6, and 144 readings and the mean of the preceding six readings. The enhanced forecast adds lags 2, 3, 12, and 36; rolling means, population standard deviations,

minima, and maxima over 6, 36, and 144 prior readings; and the change between the two most recent readings. The six-reading mean is included only once. With five calendar values, the enhanced design has 25 predictors.

All target-derived features are shifted before rolling calculations. At target position t, each window ends at t minus one. The first 144 observations cannot supply a full history and are omitted from forecast fitting, leaving 15,644 eligible development rows. During evaluation, earlier actual readings become available sequentially for subsequent predictions. Model parameters remain fixed after fitting. This is a rolling one-step evaluation with observed history, not a fixed-origin prediction of the entire final month.

IV. Methods and Evaluation

A. Chronological Development and Evaluation

The first 15,788 observations form the development period, ending April 30, 2016 at 08:10. The final 3,947 observations begin at 08:20 on the same day and end May 27 at 18:00. Both tasks are scored on these same final timestamps. Forecast history may include earlier final-period outcomes only after those outcomes would have been observed. No final-period target enters estimator fitting or hyperparameter selection.

Four expanding development folds select configurations by the arithmetic mean of fold RMSE. Each fold excludes 144 observations between fitting and validation, using TimeSeriesSplit [8]. The 144 omitted readings span one day; the first validation timestamp is 1,450 minutes after the last fitted timestamp because both endpoints are observation times. Final refitting uses all eligible development rows and the final period starts immediately afterward, without an additional gap.

The expanded study also uses three outer expanding folds. Within each outer training window, three inner temporal folds repeat the entire model-family and parameter search. Both levels use a 144-row gap. Outer validation predictions therefore evaluate the procedure that selects and fits a model, not merely a preselected estimator. Fig. 2 shows the forecast boundaries; environmental folds use all 15,788 development rows and consequently have slightly different dates. Gap readings remain available as past meter history at a later prediction time, although they are excluded from estimator fitting for that split.

The study has two experimental phases. The reference phase established the original final-period results. The expanded phase then explored richer features and additional model families using only development data for fitting and selection. Because the final period had already been examined, the expanded results on it are explicitly retrospective. Nested development validation improves internal evidence but cannot remove this history of researcher exposure or convert the final period into a new untouched test.

B. Estimators and Bounded Searches

Constant baselines predict the training-target mean or median. MLR fits an intercept and linear coefficients by least squares. StandardScaler is inside each linear pipeline, so every fold estimates its own feature means and population standard deviations from fitting rows only. Ridge uses the same preprocessing and minimizes the sum of squared residuals plus alpha times the sum of squared slope coefficients; the intercept is not penalized [4].

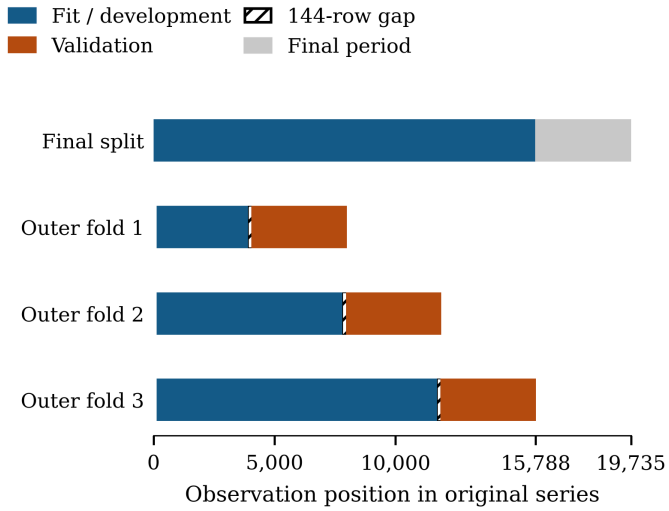


Fig. 2. Actual forecast split positions. Each outer fit repeats the three-fold inner search. The first 144 raw observations supply history but are not forecast training examples. The gap is excluded from fitting, not from subsequently observed meter history.

The reference environmental comparison includes MLR with and without calendar terms; ridge with alpha in {0.1, 10, 1000}; and histogram gradient boosting with 150 iterations, 7 or 15 maximum leaves, and L2 penalties of 1 or 10. Boosting early stopping is disabled. The reference lagged ridge uses the same three alpha values. Constant baselines are fitted separately within each training window.

The expanded environmental search evaluates 13 configurations: mean and median baselines, calendar MLR, ridge alpha values {100, 1000, 10000, 100000}, four additive-spline combinations, and two extra-trees configurations. The spline pipeline uses cubic splines with 3 or 5 knots, no bias feature, constant extrapolation, standardization, and ridge alpha of 100 or 1000. Extra trees use 100 trees, a maximum feature fraction of 0.8, and minimum leaf sizes of 20 or 60.

The expanded forecast search evaluates 17 configurations: mean, median, and persistence baselines; four history-ridge penalties {0.1, 10, 1000, 10000}; eight history-boosting combinations; and two history extra-trees settings. Boosting uses 200 iterations, learning rate 0.05, 7 or 15 maximum leaves, L2 penalties of 10 or 100, squared-error or Poisson loss, and no early stopping. Forecast extra trees uses 100 trees, feature fraction 0.8, and minimum leaf sizes of 5 or 20. Other options follow scikit-learn 1.8.0 defaults, and random-state parameters are fixed at 42.

Within each task, the configuration with the lowest mean development RMSE is selected and refitted. All candidate results remain available in the exported search tables. A lower final-period MAE does not override the declared RMSE

selection rule. The search is bounded and reproducible; it does not claim to exhaust the possible algorithms, feature sets, or hyperparameters.

C. Metrics and Diagnostic Definitions

Let e_i denote observed energy minus predicted energy at evaluation observation i, n the number of evaluated observations, and $\bar{y}$ the evaluation-target mean. The reported metrics are

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |e_i| \quad (1)$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2} \quad (2)$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n} e_i^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2} \quad (3)$$

MAE and RMSE have units of Wh per ten-minute observation. RMSE gives larger residuals greater influence. The coefficient of determination, R^2, is dimensionless, can be negative, and is not a percentage of correct predictions. A training-mean baseline can have a slightly negative R^2 because the later target mean differs from the development mean. All scores use raw predictions, including negative values from unconstrained regressors.

For calendar MLR, diagnostics include standardized coefficients, variance inflation factors (VIFs), the condition number of the standardized design with an intercept, residual autocorrelation, and errors during high-use intervals. A peak is an observed final-period target at or above the development target's 90th percentile. The threshold is fixed before evaluating the final-period peak subset. Coefficients describe conditional associations and are not interpreted as causal effects.

D. Dependence-Aware Uncertainty

Resampling individual observations would ignore serial dependence. Block resampling provides a way to retain local dependence when estimating sampling variability [9]. The implemented paired circular-block procedure samples starting positions uniformly, takes consecutive blocks with wraparound, concatenates them, and truncates each resample to the original evaluation length. The model and baseline use exactly the same resampled observation indices.

For each of 1,000 resamples, the statistic is model RMSE minus baseline RMSE. The 2.5th and 97.5th percentiles form the reported interval. The reference MLR comparison uses 144-row blocks. The expanded study repeats the comparison with 36, 144, and 288 rows, corresponding to 6, 24, and 48 hours. These are exploratory intervals conditional on fixed fitted models and the observed final period; they do not include retraining, feature-selection, or model-selection uncertainty. Approximate local stability is an assumption of this summary, not a proven property of the household series.

V. Results and Diagnostics

A. Environmental Estimation

Table I combines the environmental results without duplicating recipes shared by the two phases. Calendar MLR lowers final-period RMSE from 86.51 to 85.69 Wh relative to environmental-only MLR. The reference-selected ridge, with alpha 1000, reaches 84.71 Wh and R^2 of 0.134. These improvements are modest: most variation remains unexplained under this information constraint. The median baseline has the lowest MAE in this comparison, illustrating why metric choice changes the apparent ranking.

TABLE I
ENVIRONMENTAL ESTIMATION ON THE FINAL PERIOD

Model	MAE (Wh)	RMSE (Wh)	R^2
Mean baseline	52.685	91.053	-0.0003
Median baseline	43.278	98.037	-0.160
MLR environment	51.155	86.509	0.097
MLR calendar	46.933	85.686	0.114
Ridge calendar (a)	47.222	84.708	0.134
Reference boosting	97.829	125.878	-0.912
Additive splines	64.644	91.613	-0.013
Extra trees (b)	61.365	91.187	-0.003

All 3,947 final-period observations. (a) Reference CV winner, alpha = 1000; also the best expanded ridge setting. (b) Expanded CV winner, minimum leaf size = 60. Expanded-phase results are retrospective. Raw predictions are scored.

The reference histogram boosting model reaches RMSE 125.88 Wh and R^2 of −0.912. This is a measured failure to transfer under the specified configuration, rather than a failed execution. It does not show that nonlinear models are generally unsuitable. The expanded search tests this possibility more broadly while preserving the earlier outcome.

In the expanded environmental search, extra trees with minimum leaf size 60 have the lowest development RMSE, 92.25 Wh, compared with 93.60 Wh for ridge. On the previously examined final period, however, extra trees reached 91.19 Wh, worse than the reference ridge. Additive splines reach 91.61 Wh. The environmental search therefore does not improve the retrospective result. Keeping ridge as the interface default is a practical continuity choice; it does not make ridge the winner of the expanded cross-validation search.

B. Rolling Forecasting

Table II reports forecasting on the same 3,947 timestamps. Persistence predicts the most recent observed energy and reaches RMSE 66.44 Wh. The previous-day baseline reaches 112.18 Wh, indicating that a matching clock time one day earlier is insufficient for these intervals. The reference lagged ridge improves RMSE to 60.99 Wh, while persistence has slightly lower MAE than that reference model.

TABLE II
ROLLING ONE-STEP FORECASTING ON THE FINAL PERIOD

Model	MAE (Wh)	RMSE (Wh)	R^2
Mean baseline	52.659	91.052	-0.0003
Median baseline	43.278	98.037	-0.160
Persistence	26.651	66.438	0.467
Previous day	53.679	112.177	-0.518
Reference lagged ridge	26.897	60.994	0.551
History ridge	27.659	59.695	0.570
History boosting	25.307	58.001	0.594
History extra trees (a)	25.506	57.230	0.605

All 3,947 final-period observations. (a) Expanded CV winner. Expanded-phase results are retrospective. The forecast training mean differs slightly from the environmental training mean because 144 initial history rows are unavailable.

With the richer history representation, ridge improves RMSE to 59.69 Wh, histogram boosting to 58.00 Wh, and extra trees to 57.23 Wh. The selected history extra-trees model has 100 trees, minimum leaf size 5, and feature fraction 0.8. Its MAE is 25.51 Wh and R^2 is 0.605. Relative to persistence, the RMSE reduction is 9.21 Wh, or 13.86 percent; relative to the reference lagged ridge, it is 3.76 Wh, or 6.17 percent.

History boosting has slightly lower MAE, 25.31 Wh, but the declared RMSE criterion selects extra trees. These results describe alternative error tradeoffs rather than a model that is best under every loss. The improved forecast also has a different information set from environmental estimation. Its lower error cannot be attributed solely to a more expressive learning algorithm.

Fig. 3 compares the first 288 final-period predictions, a fixed 48-hour sequence selected by position rather than visual performance. The environmental ridge misses abrupt changes, while observed-history forecasting tracks more of the short-term variation. The plot also retains large errors and isolated peaks. It illustrates behavior in one window and does not replace the full-period metrics.

C. Nested Validation and Interval Sensitivity

Table III reports outer-fold results for the expanded selection procedure. Each forecast fold selects history extra trees and improves RMSE over its persistence baseline. The selected minimum leaf sizes are 5, 20, and 5 across folds, showing that nested evaluation repeats parameter selection rather than fixing the final-period recipe in advance. The environmental procedure selects ridge with alpha 10000 in the first outer fold and extra trees with leaf size 60 in the other two; each improves on its training-mean baseline within that fold.

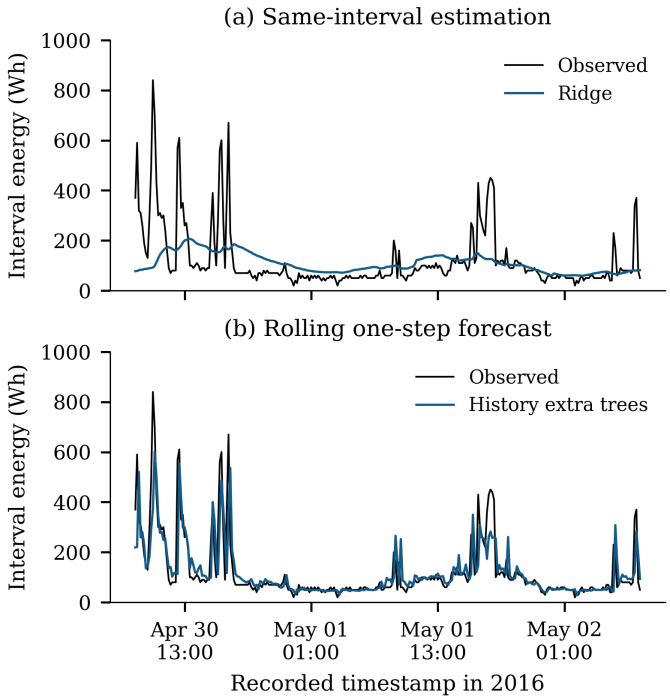


Fig. 3. The first 288 final-period observations, without smoothing or aggregation. Both panels share the same energy scale. Forecast inputs use actual readings available before each target interval; the environmental panel uses recorded same-interval conditions.

TABLE III
OUTER TEMPORAL VALIDATION OF MODEL SELECTION

Task / fold	Selected family	RMSE (Wh)	Baseline (Wh)
Env. 1	Ridge	96.510	99.202
Env. 2	Extra trees	89.897	99.658
Env. 3	Extra trees	92.643	103.217
Forecast 1	History extra trees	66.618	72.361
Forecast 2	History extra trees	61.331	70.863
Forecast 3	History extra trees	60.925	72.279

Baselines are the training mean for environmental estimation and persistence for forecasting. Each outer fold repeats the full three-fold inner search. Environmental and forecast validation windows differ after history warm-up.

The environmental outer results and the poorer final-period extra-trees score are compatible with time-varying model behavior. They are not evidence that one split must be incorrect. Forecast outer validation windows each contain 3,911 observations; environmental windows each contain 3,947. Comparisons should therefore be made within a task and fold. No independent-fold significance test is applied, because the expanding training sets overlap and the series is temporally dependent.

Fig. 4 reports the expanded paired bootstrap intervals. For history extra trees minus persistence, all three intervals lie below zero. With 24-hour blocks, the interval is [−11.35, −6.88] Wh. For environmental extra trees minus the mean baseline, every interval includes zero; the corresponding

24-hour interval is [−7.87, 9.15] Wh. This supports a forecast advantage in the observed period under the resampling assumptions, but not a reliable environmental advantage for the expanded winner.

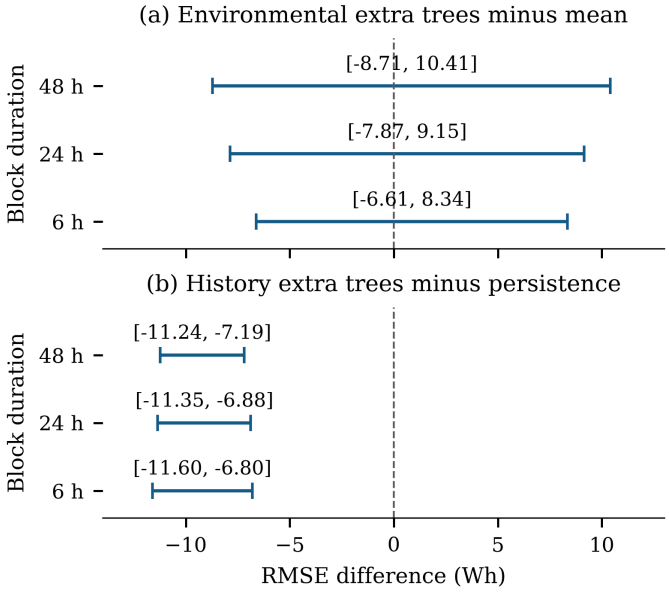


Fig. 4. Paired 95 percent percentile intervals for model-minus-baseline RMSE, using 1,000 circular-block resamples per block length. Negative values favor the model. Lines show interval endpoints only, not independent estimates or selection-adjusted confidence intervals.

D. Residual Structure and Interpretation

Calendar MLR residuals have lag-one correlation 0.691. The development-defined peak threshold is 210 Wh, and 325 final-period observations meet it. Their MAE is 225.73 Wh, compared with 30.89 Wh for the other 3,622 observations. Fig. 5 makes the remaining dependence and concentration of error visible. A model can have a useful average improvement and still be unreliable for the high-use intervals that matter to a particular application.

The standardized design with an intercept has condition number 62.83. The largest VIFs are 237.74 for outdoor station temperature, 166.26 for dew point, and 80.00 for outdoor relative humidity. These values describe substantial predictor redundancy. Standardization makes coefficient scales comparable but does not remove collinearity. The full coefficient and VIF tables are retained for inspection; coefficient magnitude alone is not used to rank causal importance.

Calendar MLR produces 48 negative raw predictions, and reference ridge produces three. These values are physically implausible but remain in the evaluation to expose the limitation of unconstrained regression. The convenience interface separately reports nonnegative values, without substituting them into the benchmark. The reference calendar-MLR-minus-mean bootstrap interval is [−8.67, −1.74] Wh at 144-row blocks. It concerns the reference comparison and should not be conflated with the expanded environmental result in Fig. 4.

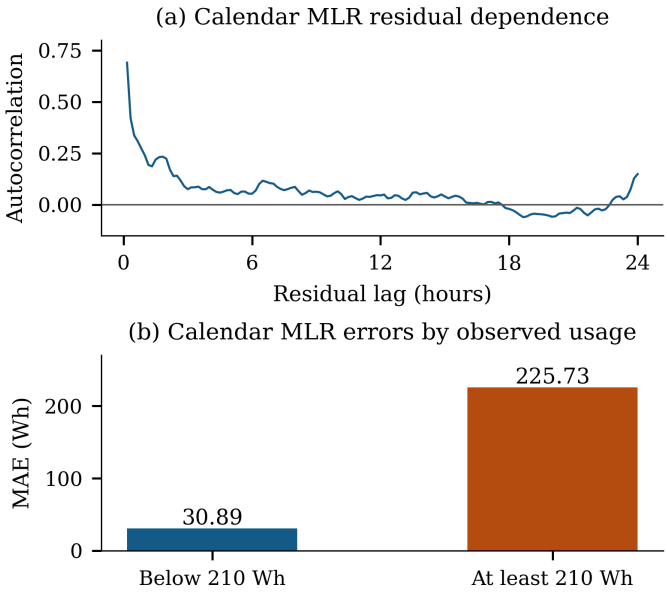


Fig. 5. Calendar MLR diagnostics over all 3,947 final-period observations. The peak threshold is the development 90th percentile. Panel (a) is descriptive autocorrelation, without independent-error significance bands; panel (b) compares 3,622 nonpeak and 325 peak observations.

VI. Implementation and Reproducibility

A. Shared Analysis and Inference

WattWise.ipynb is the primary analysis interface. The notebook and command-line experiment runners call the same source modules, avoiding a separate notebook-only implementation. src/energy.py implements validation, features, reference experiments, metrics, and diagnostics. src/enhanced.py implements the expanded search, nested validation, history features, and uncertainty analysis. src/inference.py validates prediction inputs and fits the declared recipes on development observations.

The artifact includes the unchanged raw data, provenance, an executed notebook, all candidate search scores, selected configurations, per-observation predictions, nested-fold predictions and boundaries, daily error summaries, coefficients, VIFs, and figures. CSV tables support numerical inspection; JSON records preserve configuration, versions, and checksums. A GitHub Actions workflow supplies automated checks, while the README documents setup and interpretation. Software and documentation use the MIT license; the dataset retains its original attribution and CC BY 4.0 terms.

The condition-estimation command accepts recorded environmental covariates and timestamps, then outputs raw energy, a separately clipped nonnegative value, and an outside-training-range flag. The default recipe remains reference ridge. The --experimental-v2 option requests the development-selected environmental extra-trees recipe. Range flags only identify feature-wise training-range violations; they are neither prediction intervals nor a calibrated distribution-shift detector.

The forecasting command requires at least 144 contiguous prior readings with date and Appliances columns, and predicts the timestamp ten minutes after the last reading. Validation rejects missing, infinite, negative, duplicate, or discontinuous history. The selected recipe and parameter names are checked against the declared search, the model is refitted from development observations, and the program does not load an arbitrary serialized estimator. Both interfaces prevent an output path from overwriting its input and create missing output directories.

B. Executed Tests and Evidence Audit

The final manuscript audit reran the complete 49-test suite successfully on November 10, 2026, with no failures. Tests include actual-data recipe fits, feature exclusions, temporal split boundaries, training-only scaling, known metric calculations, recomputation from saved predictions, source provenance, malformed-input rejection, command-line consistency, and overwrite protection. Synthetic arrays appear only as small arithmetic or invalid-input fixtures; all research model training uses the real UCI observations.

Leakage checks change current and future targets and verify that earlier history features remain identical. Separate tests verify that changing final-period targets cannot change the fitted environmental predictor. Interface tests compare computed values with the corresponding saved predictions. These checks test concrete invariants; they do not prove every possible operational use is leakage-free or that predictions generalize to unseen dwellings.

Earlier auditing reproduced three defects before fixing them: an empty table with headers could pass loading; source hashing could refer to the default file instead of the loaded file; and forecast splitting could depend on numeric index labels. Regression tests for those cases now pass. The current loader carries the actual source-file hash, the manifest separately records a hash of the analyzed frame, and forecast routines reset row indices before positional splitting.

For this manuscript, the reference experiments, expanded searches, three-by-three nested validation, diagnostics, and bootstrap calculations were rerun from the bundled data. Twenty-three archived result artifacts matched the fresh results with relative tolerance 10^{-9} and absolute tolerance 10^{-8}; runtime columns were excluded because elapsed time changes between runs. The audit preserves its log and comparison record. Tables are populated from the result files, and the five figures are regenerated from data or predictions with explicit checks of counts, dates, boundaries, and plotted ranges.

The notebook artifact contains 13 previously executed code cells and no error outputs, and its schema was revalidated during this audit. Its recorded execution used a fresh IPython process because the earlier host blocked ordinary kernel socket setup. The present document audit reran the underlying experiments, rather than claiming a new native-kernel execution. Windows, macOS, external GitHub Actions execution, and another household have not been independently verified.

C. Reproduction Procedure

The verified core environment is Python 3.12.14, NumPy 2.3.5, pandas 2.2.3, and scikit-learn 1.8.0. Install requirements.txt in a virtual environment before running the project from its root directory. The raw dataset is bundled, so experiments need no network after dependency installation. The optional download script restores checksum-matched source data. Core packages are pinned, but a complete transitive dependency lockfile is not supplied; small numerical differences may occur across numerical-library builds.

The sequence python run_experiments.py followed by python run_enhancements.py regenerates the reference and expanded results. Run python -m pytest -q to execute the tests. The notebook may be opened in JupyterLab; scripts/execute_notebook.py also provides an --in-process option for environments that block kernel sockets. The audit command python scripts/audit_ieee_evidence.py recomputes the experiments and compares them with the saved evidence. Generated outputs should never be manually edited to change a score.

For condition estimation, run python predict.py data/example_conditions.csv results/new_predictions.csv. For forecasting, run python forecast.py data/example_history.csv results/next_interval.csv. The examples use actual observations. Model or protocol changes require regenerating the evidence and reviewing the manuscript together; further model development should use a newly reserved evaluation period rather than repeatedly optimizing against the current final window.

VII. Limitations and Future Validation

The most important limitation is the scope of the data. One dwelling and partial seasonal coverage do not establish performance across housing types, occupancy patterns, appliance inventories, or a full year. Temporal validation exposes some changes over time but cannot replace external validation. The final-period reuse in the expanded study further limits confirmatory interpretation, even though estimator fitting and numerical selection remain restricted to development observations.

Environmental estimation relies on recorded covariates, including interpolated weather, and therefore does not establish a deployable forecast under unknown future weather. The history model assumes the preceding energy reading is available before the next prediction. Meter delays, missing readings, irregular timestamps, or longer forecast horizons would change that information contract and require new experiments. Rejecting invalid inputs is a reproducibility safeguard, not a complete operational strategy for sensor outages.

Uncertainty intervals are conditional on particular fitted models and a short observed series. They omit uncertainty due to refitting, researcher choices, and household sampling. Circular wraparound also treats the end and beginning of the evaluated sequence as adjacent for resampling. The block-length comparison is a sensitivity check, not proof that the chosen lengths capture all dependence. No causal savings,

calibrated individual prediction intervals, billing accuracy, or autonomous control capability is demonstrated.

The next study should reserve a genuinely new period or household, define meter-arrival latency in advance, and evaluate multiple horizons. Occupancy or appliance-state information may explain switching events that environmental sensors miss, but any such features require their own availability and privacy assessment. A deployment study would also need monitoring, missing-data handling, latency and resource measurements, and an application-specific cost of missed peaks. The present bounded search is a reproducible starting point for those studies.

VIII. Conclusion

WattWise shows a modest temporal benefit from environmental MLR and ridge regression, a stronger benefit from recent energy history in a distinct one-step forecast, and a negative result from expanding environmental model complexity. Reference ridge attains RMSE 84.71 Wh for same-interval estimation. History extra trees attain 57.23 Wh for rolling forecasting, with consistent improvements over persistence across three nested development folds. The expanded final-period comparisons remain retrospective.

The accompanying software records model configurations, per-observation predictions, diagnostic tables, and reproducible figures, together with prediction interfaces and a verification suite of 49 passing tests. These materials support independent examination and reproduction of the reported findings. Stronger claims now depend on fresh external observations and a specified operational prediction contract, rather than further tuning on the same evaluation period.

Data and Code Availability

The public dataset and originating analysis are available through [1] and [3]. The accompanying WattWise project archive contains the Python source, notebook, data attribution, results, manuscript source, figure builders, and audit records, are available in project's Github Repository [10].

References

[1] L. Candanedo, "Appliances Energy Prediction," UCI Machine Learning Repository, 2017, dataset, doi: 10.24432/C5VC8G.

[2] L. M. Candanedo, V. Feldheim, and D. Deramaix, "Data driven prediction models of energy use of appliances in a low-energy house," *Energy and Buildings*, vol. 140, pp. 81–97, 2017, doi: 10.1016/j.enbuild.2017.01.083.

[3] L. M. Candanedo, "Appliances-energy-prediction-data," GitHub repository. Accessed: Nov. 10, 2026. [Online]. Available: https://github.com/LuisM78/Appliances-energy-prediction-data.

[4] scikit-learn developers, "Ridge," scikit-learn 1.8.0 documentation. Accessed: Nov. 10, 2026. [Online]. Available: https://scikit-learn.org/1.8/modules/generated/sklearn.linear_model.Ridge.html.

[5] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006, doi: 10.1007/s10994-006-6226-1.

[6] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, Art. no. 91, 2006, doi: 10.1186/1471-2105-7-91.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html.

[8] scikit-learn developers, "TimeSeriesSplit," scikit-learn 1.8.0 documentation. Accessed: Nov. 10, 2026. [Online]. Available: https://scikit-learn.org/1.8/modules/generated/sklearn.model_selection.TimeSeriesSplit.html.

[9] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989, doi: 10.1214/aos/1176347265.

[10] [10] S. Shakya, "WattWise: Household appliance energy estimation and forecasting," version 2.0.0, GitHub repository, 2026. Accessed: Nov. 10, 2026. [Online]. Available: https://github.com/requiemdoomed/WattWise